Présentation de :



Apache Airflow

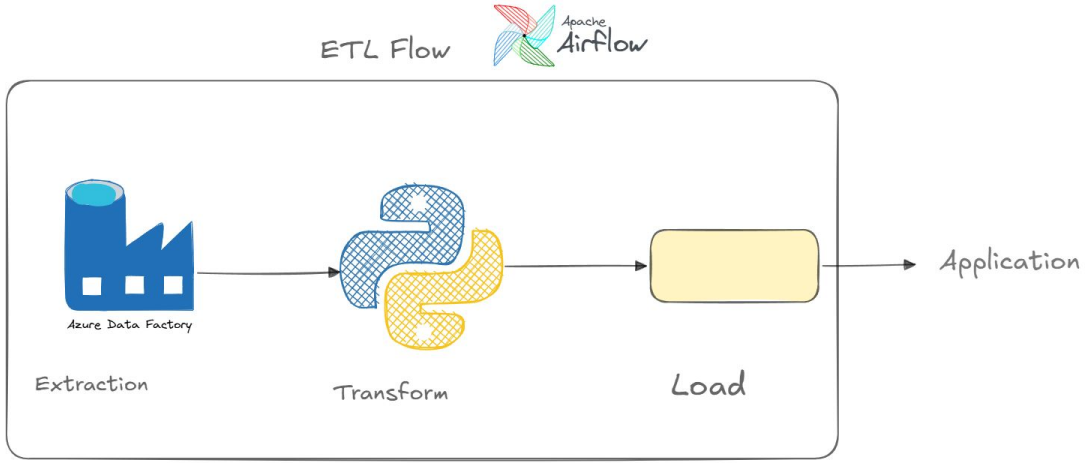
Veille technique par Frédéric, Romain et Tesfa

Plan

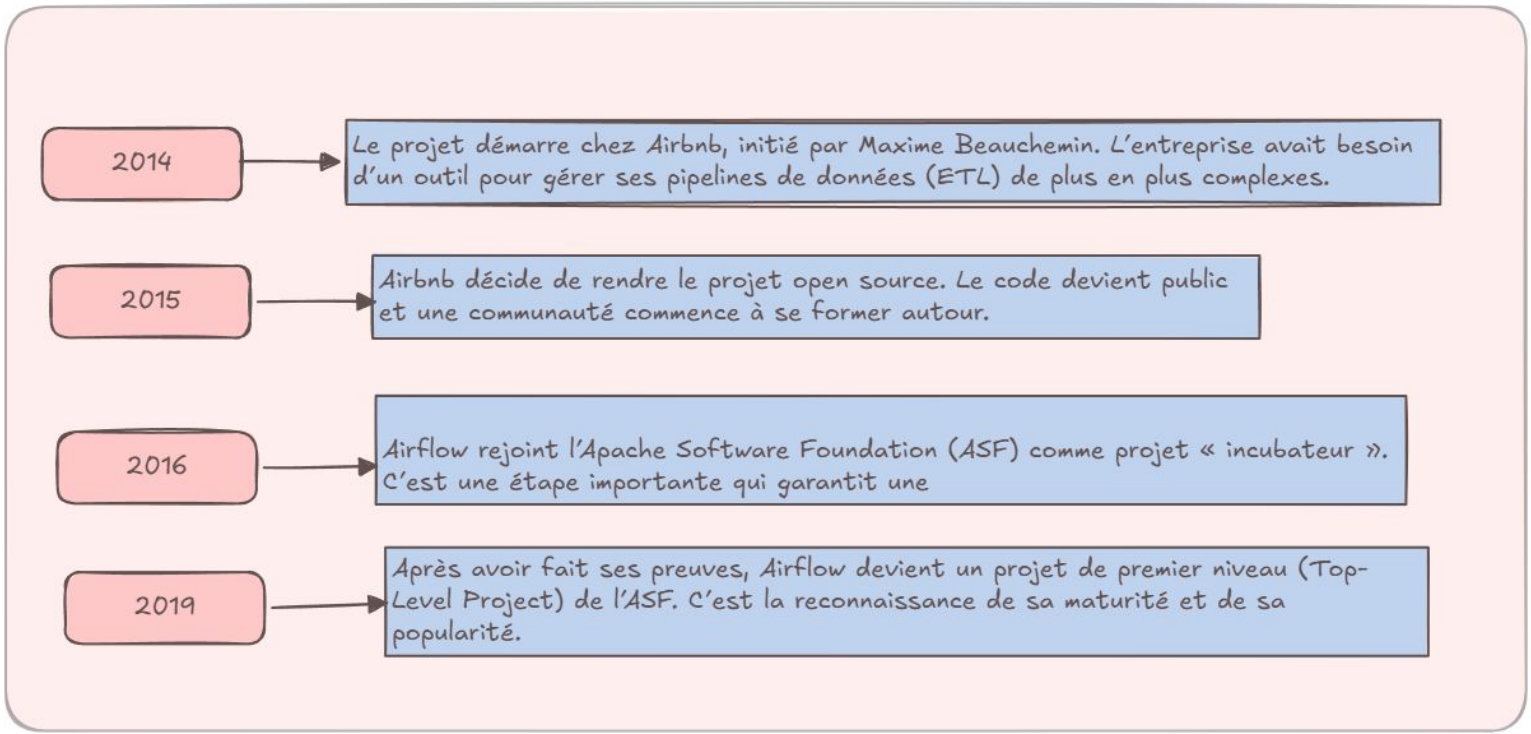
- Airflow c'est quoi ?
- Pourquoi Airflow (cas d'usage) ?
- Concepts clés : DAG, opérateurs, tâches
- Base de données de métadonnées
- Architecture
- Interface utilisateur et monitoring
- Création et gestion des DAGs
- Principaux opérateurs
- Programmation et planification des tâches
- Gestion des erreurs
- Avantages et limites / alternatives
- Airflow pour le projet TradeCorp

Airflow c'est quoi ?

Apache Airflow est une plateforme open source pour créer, planifier et superviser des workflows



Histoire


2014
Le projet démarre chez Airbnb, initié par Maxime Beauchemin. L'entreprise avait besoin d'un outil pour gérer ses pipelines de données (ETL) de plus en plus complexes.
2015
Airbnb décide de rendre le projet open source. Le code devient public et une communauté commence à se former autour.
2016
Airflow rejoint l'Apache Software Foundation (ASF) comme projet « incubateur ». C'est une étape importante qui garantit une
2019
Après avoir fait ses preuves, Airflow devient un projet de premier niveau (Top-Level Project) de l'ASF. C'est la reconnaissance de sa maturité et de sa popularité.

Pourquoi Airflow ?

Airflow sert à **orchestrer, planifier** et **surveiller** des pipelines de données complexes.

🚀 **Cas d'usage concrets (réels en entreprise)**

1. Orchestration de pipelines ETL/ELT
2. Planification (scheduler)
3. Orchestration de jobs Spark
4. Monitoring + Alerting
5. Gestion des dépendances
6. Intégration multi-technologies
7. Reproductibilité + versioning
8. Data Quality

Pourquoi Airflow plutôt qu'un simple script Python ?



Parce qu'un script Python ne gère pas :

- Les dépendances
  - les retries
  - les logs centralisés
  - La planification
  - La visualisation
  - Les alertes
  - L'orchestration multi-technologies
  - Les environnements distribués

Airflow est conçu pour les pipelines de production, pas pour les scripts artisanaux.

Concepts clés : DAG, opérateurs, tâches

DAG (Directed Acyclic Graph)

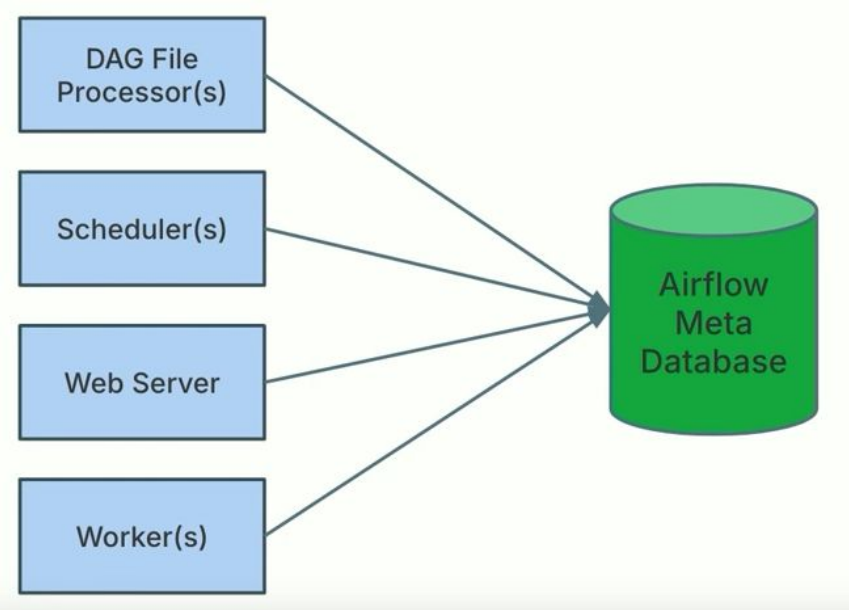
Ce qu'un DAG définit :

- les tâches (steps du pipeline)
- les dépendances (A → B → C)
- la planification (tous les jours à 6h)
- le contexte d'exécution (paramètres, variables, connexions)

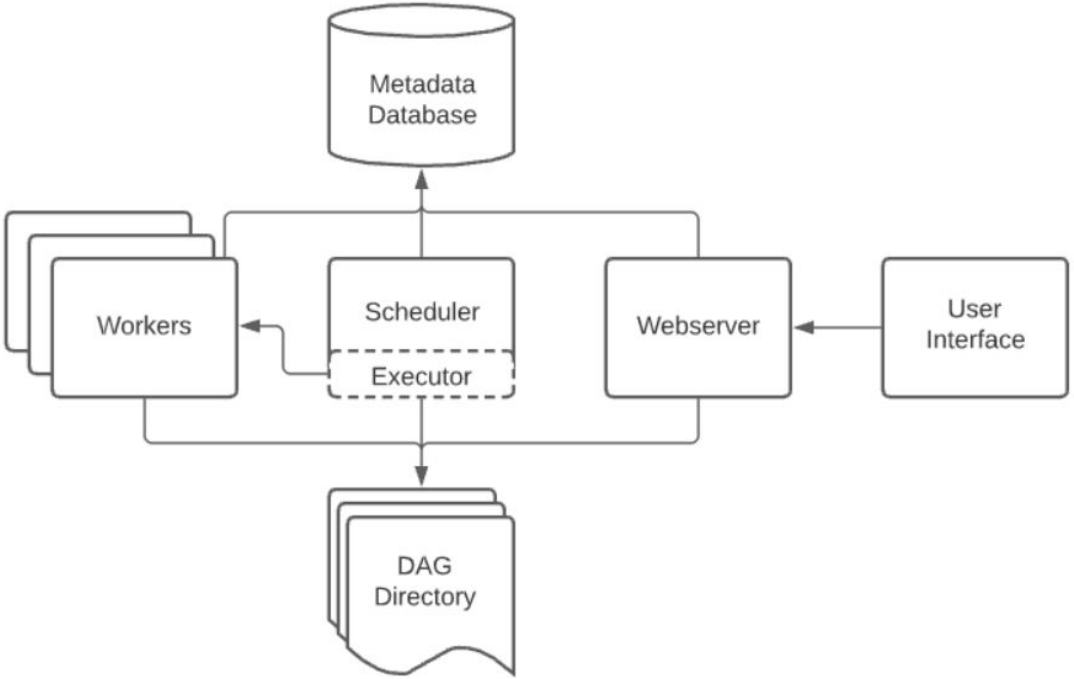
Base de données de métadonnées

Une *base de données de métadonnées* stocke des informations sur les permissions utilisateurs, les DAG passés et actuels et les exécutions de tâches, les configurations des DAG, et plus encore. Il sert de source de vérité pour le planificateur.

Par défaut, Airflow gère les métadonnées avec SQLite, qui est destiné uniquement au développement



Architecture



Un **planificateur**:

- déclenchement des workflows planifiés
- soumission des tâches à l'exécuteur

Un **exécuteur**:

- l'exécution des tâches
- en production envoient l'exécution des tâches aux workers.

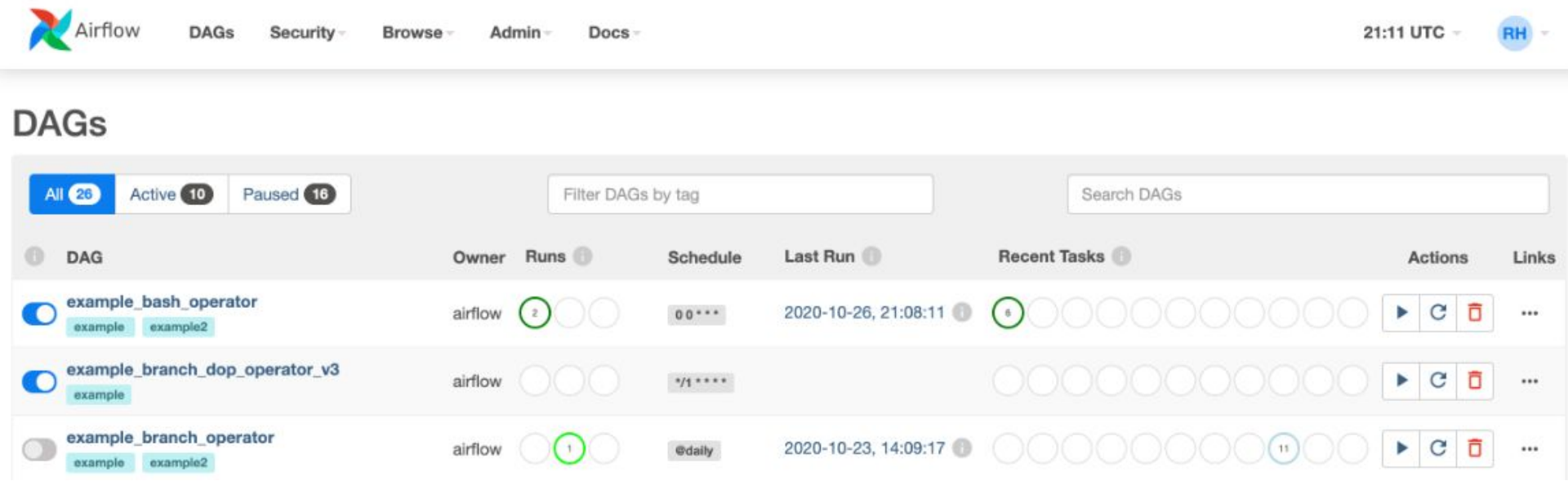
Un **serveur web**, interface pratique pour:

- inspecter
- déclencher
- déboguer les DAGs et des tâches

Un **répertoire DAG**, lu par le planificateur et l'exécuteur, et par tous les workers

Une **base de données de métadonnées**, utilisée par le planificateur, l'exécuteur et le serveur web pour stocker l'état.

Interface utilisateur et monitoring



Gestion des DAGs, de leur programmation, des états, de leurs exécutions

Création et gestion des DAGs

1. **Importer les modules nécessaires** : Importer DAG depuis Airflow, les modules de gestion du temps comme datetime et timedelta, ainsi que les opérateurs spécifiques aux tâches (ex: BashOperator, PythonOperator).
2. **Définir les arguments par défaut** : Configurer les paramètres communs à toutes les tâches, notamment la start_date et la politique de retry.
3. **Créer l'objet DAG** : Instancier airflow dag avec un identifiant unique et un intervalle de planification précis.
4. **Définir les tâches** : Utiliser les opérateurs appropriés pour créer chaque étape du workflow.
5. **Établir les dépendances** : Structurer l'ordre d'exécution avec la syntaxe tache1 >> tache2 qui indique que tache2 dépend de la réussite de tache1.

Principaux opérateurs

Opérateur	Fonction principale	Cas d’usage typique
PythonOperator	Exécute une fonction Python	Logiques personnalisées, transformations de données
BashOperator	Exécute des commandes shell	Scripts système, commandes simples
HttpOperator	Effectue des requêtes HTTP	Intégration d’API, webhooks
SQLOperator	Exécute des requêtes SQL	Opérations sur bases de données
DockerOperator	Lance des conteneurs Docker	Isolation d’environnement
Sensors	Attente d’événements externes	Vérification de disponibilité de fichiers/services

Programmation et planification des tâches

- **Planification** (schedule) : déclenchement automatique selon un rythme défini (@daily, @hourly, ou cron précis)
- **Date de début** (start_date) : date à partir de laquelle le DAG commence à s'exécuter
- **Reprise** (catchup) : si True, rejoue automatiquement tous les runs manqués depuis start_date
- **Dépendances** : chaînage via >> : reader >> transformer >> writer

```
with DAG("tradecorp_etl",
         schedule="@daily",
         start_date=datetime(2026, 1, 1),
         catchup=False) as dag:
    reader() >> transformer() >> writer()
```

Chaque tâche ne démarre que quand la précédente a réussi.

Gestion des erreurs & Robustesse

- **Backfilling** *(dag)* : rejoue automatiquement les pipelines sur des périodes passées (catchup=True), manuel via CLI
- **Retries** *(dag & task)* : relance les tâches échouées avec délai configurable (retries=3, retry_delay=5min)
- **SLA** *(task)* (Service Level Agreement) : définit un temps max d'exécution ; alerte si non respecté (sla=timedelta(hours=2))
- **Alerting** *(dag & task)* : notifications automatiques en cas d'échec ou de succès (via mail, slack, teams...)

Avantages et Limites

Avantages

- Open-source, licence Apache 2.0.
- Communauté très active (derniere maj aout 26), standard de l'industrie.
- « Workflows as Code » en Python : versionnables, testables.
- UI web riche : DAGs, runs, logs, XComs, code.
- Écosystème de providers (> 300 intégrations).
- Scalable horizontalement (+ de noeuds)
- Backfilling natif, retries, SLA, alerting.

Limites

- Non conçu pour le streaming temps réel.
- Setup en production complexe (plusieurs composants).
- Courbe d'apprentissage : notions de DAG, executor, XCom (Cross-Communications)…
- Consommation de ressources si beaucoup de sensors en mode polling.
- Dépendances Python parfois lourdes à gérer.
- UI pas toujours fluide sur de très gros déploiements.
- Historique des runs peut alourdir la DB des métadonnées.

Alternatives

- **Prefect + Dagster** *(rachat juillet 2026)* : Orchestration en Python et gestion par assets, désormais réunis pour rivaliser avec Airflow
- **Kestra** : Orchestration déclarative en YAML, multi-langages, scalable horizontalement, interface low-code
- **Argo Workflows** : Natif Kubernetes, chaque tâche s'exécute dans un conteneur éphémère, montée en charge par ajout de nœuds
- **Temporal** : Exécution fiable et durable de workflows événementiels, pensée pour les microservices

Airflow pour le projet TradeCorp

Airflow orchestre notre ETL PySpark en un DAG planifié :

Reader (ADLS) → Transformer (PySpark) → Writer (Parquet/ADLS)

1. **Planification** — exécution automatique selon un cron
2. **Enchaînement** — chaque jalon attend la fin du précédent
3. **Reprises** — retries auto en cas d'échec
4. **Monitoring** — UI web pour suivre l'état et les logs

Airflow n'exécute pas PySpark — il déclenche, ordonne et surveille les trois jalons.

Références

1. Apache Airflow: Qu’est-ce que c’est et à quoi ça sert?
2. Airflow : Guide Complet pour Orchestration et Workflows Automatisés
3. Apache Airflow : qu'est-ce que c'est et comment l'utiliser ?
4. Architecture Overview — Airflow Documentation
5. Le bon et le mauvais de l’orchestration du pipeline Airflow Apache